Laboratorio 12 – Problema 1

1. Reducción-β de la operación lógica NOT

Partiendo de la codificación de Church para booleanos, NOT se define invirtiendo el orden en que un booleano recibe sus dos ramas:

```
TRUE  ≡ λx.λy.x
FALSE ≡ λx.λy.y
NOT   ≡ λb.λt.λf. b f t
```

Reducción de NOT aplicado a TRUE:

```
  NOT TRUE
= (λb.λt.λf. b f t) TRUE
→β λt.λf. TRUE f t                    [se sustituye b := TRUE]
= λt.λf. (λx.λy.x) f t
→β λt.λf. (λy.f) t                     [se sustituye x := f]
→β λt.λf. f                            [se sustituye y := t]
= FALSE   □
```

Reducción de NOT aplicado a FALSE (verificación simétrica):

```
  NOT FALSE
= (λb.λt.λf. b f t) FALSE
→β λt.λf. FALSE f t                   [se sustituye b := FALSE]
= λt.λf. (λx.λy.y) f t
→β λt.λf. (λy.y) t                    [se sustituye x := f]
→β λt.λf. t                            [se sustituye y := t]
= TRUE    □
```

En ambos casos se aplica la misma regla de β-reducción: (λx.M) N → M[x := N], es decir, se sustituye toda ocurrencia libre de la variable ligada por el argumento aplicado. Dado que los nombres de las variables ligadas en TRUE/FALSE (x, y) no coinciden con los de NOT (b, t, f), no hay riesgo de captura de variables y por lo tanto tampoco hace falta α-conversión (renombrar variables) antes de sustituir. Así, el resultado confirma que NOT funciona correctamente: al recibir un booleano b y las dos ramas t/f, NOT simplemente se las entrega a b en orden invertido, y como un booleano de Church selecciona su primer argumento si es TRUE y su segundo si es FALSE, invertir ese orden invierte el resultado final.

2. Recursión y ciclos en cálculo lambda / programación funcional

Ciclos (for / while)

El cálculo lambda puro no tiene ninguna construcción de ciclo: no existe for ni while. Esto es consecuencia directa de que no hay variables mutables ni estado, ya que un ciclo imperativo depende de reasignar una variable contador o acumulador en cada iteración, mientras que en cálculo lambda las variables solo se ligan mediante abstracción (λ) y se sustituyen, nunca se reasignan. Por esta razón, los lenguajes funcionales puros como Haskell tampoco tienen for/while como construcciones primitivas: en su lugar, toda iteración se expresa mediante recursión o mediante combinadores de orden superior construidos sobre ella, como map, filter y fold/reduce. Los lenguajes multiparadigma como JavaScript sí ofrecen ciclos, pero el estilo funcional prefiere evitarlos en favor de funciones como `.map()`, `.filter()` y `.reduce()`, que no son más que recursión "empaquetada".

Recursión

Dado que en cálculo lambda las funciones son anónimas, una abstracción λx.M no tiene nombre propio dentro de su cuerpo y, en consecuencia, no puede llamarse a sí misma directamente como ocurriría en un lenguaje con funciones con nombre. Para resolver esto se recurre al combinador de punto fijo, por ejemplo el combinador Y:

```
Y ≡ λf. (λx. f (x x)) (λx. f (x x))
```

Y cumple la propiedad Y f = f (Y f): al aplicarlo a una función f que recibe "a sí misma" como parámetro, genera una versión recursiva de f sin que f necesite referirse a su propio nombre. Así, con este combinador se puede definir, por ejemplo, el factorial:

```
FACT ≡ Y (λfact.λn. SI (ESCERO n) 1 (MULT n (fact (PRED n))))
```

En lenguajes funcionales reales como JavaScript o Haskell este rodeo ya no es necesario, puesto que el lenguaje sí permite nombrar funciones y que se refieran a sí mismas (`function fact(n) {...}`). Sin embargo, el principio de fondo sigue siendo el mismo que en el cálculo lambda puro: al no existir estado mutable ni ciclos, toda iteración se expresa como una función que se vuelve a invocar a sí misma hasta alcanzar un caso base que la detiene, lo cual equivale a la condición de salida de un ciclo.

3. ¿Cuándo es prudente usar programación funcional?

Cuándo sí conviene

Es prudente usar programación funcional cuando el problema es naturalmente una transformación de datos (pipelines de map/filter/reduce), cuando se necesita paralelismo o concurrencia segura -ya que al no haber estado mutable compartido no hay condiciones de carrera-, y cuando se valora especialmente la facilidad de prueba, pues las funciones puras son fáciles de testear al depender su salida solo de su entrada.

Ejemplo: un pipeline de procesamiento de transacciones financieras que llegan en streaming, donde primero se filtran las transacciones inválidas, luego se transforma cada una a un formato normalizado y, finalmente, se reduce para obtener totales por categoría. Como estas funciones son

puras, el pipeline puede repartirse fácilmente entre varios hilos o nodos sin riesgo de corromper datos compartidos, y además cada etapa puede probarse de forma aislada.

Cuándo no conviene

En cambio, no es prudente cuando el rendimiento depende de mutación de estado en el lugar (in-place) y de control fino sobre memoria, ya que las estructuras inmutables de la programación funcional suelen implicar copias adicionales y mayor presión sobre el recolector de basura. Tampoco conviene forzarla cuando el dominio del problema es inherentemente secuencial y con mucho estado compartido que cambia constantemente, pues modelarlo como funciones puras termina añadiendo complejidad accidental en lugar de reducirla.

Ejemplo: el bucle principal de un motor de videojuego o de un sistema embebido en tiempo real, que debe actualizar cientos de miles de posiciones, colisiones y buffers de memoria 60 veces por segundo con la menor sobrecarga posible. En ese contexto, un enfoque imperativo con arreglos mutables y actualización in-place resulta más eficiente y más simple de razonar que recrear estructuras inmutables en cada frame.